# Pokemon_Proj

## Load Necessary Packages

```
library(RMySQL) ## May also need odbc package
library(gtExtras) ## For making pretty tables
library(scales) ## Allows dollar ($) formatting
library(dplyr) ## Facilitates data manipulation
library(readr) ## Facilitates reading tabular into R
library(ggplot2) ## Fancy graphics
library(reticulate) ## Allows switching between R and Python

use_virtualenv("r-reticulate", required = TRUE)
#initializes my python enviroment for later
```

## Connect to Database

The following code uses dbConnect to connect to the Pokemon database on the statdb.byu.edu host.

```
mydb <- dbConnect(MySQL(), user='stat286', password='Cougars24!', dbname='Pokemon', host='statdb.l
```

## Instructions 1 & 2: Select Random Trainer and 6 Pokemon

The following code selects one trainer at random and six pokemon according to a specific process.

```
## Select one trainer at random from the Trainers table ## add code ##
query.trainer <- "SELECT * FROM Trainers ORDER BY RAND() LIMIT 1"
selected_trainer <- dbGetQuery(mydb, query.trainer)

## Make a list of Pokemon from the pokemon table with type = "bug" and speed > 55. Order the list
query.Pokemon <- "SELECT * FROM Pokemon WHERE type1 = 'bug' and speed > 55 ORDER BY defense, name
selected_pokemon <- dbGetQuery(mydb, query.Pokemon)
```

The following code creates data tables all_pokemon and all_trainers.

```
all_pokemon <- dbReadTable(mydb, "Pokemon")
all_trainers <- dbReadTable(mydb, "Trainers")
```

## Instruction 3: Assign random levels to Pokemon

The following code generates pokemon levels from a uniform distribution between 30 and 100 for each of your six selected pokemon.

```
set.seed(42)  # Set seed for reproducibility (removed after testing)

selected_pokemon$Level <- round(runif(6, min=30,max=100))
## complete the runif function ##
```

## Instruction 4: Query Pokémart data and output to .csv file

```
# Query PokéMart data
pokemart_df <- dbReadTable(mydb, "PokeMart")

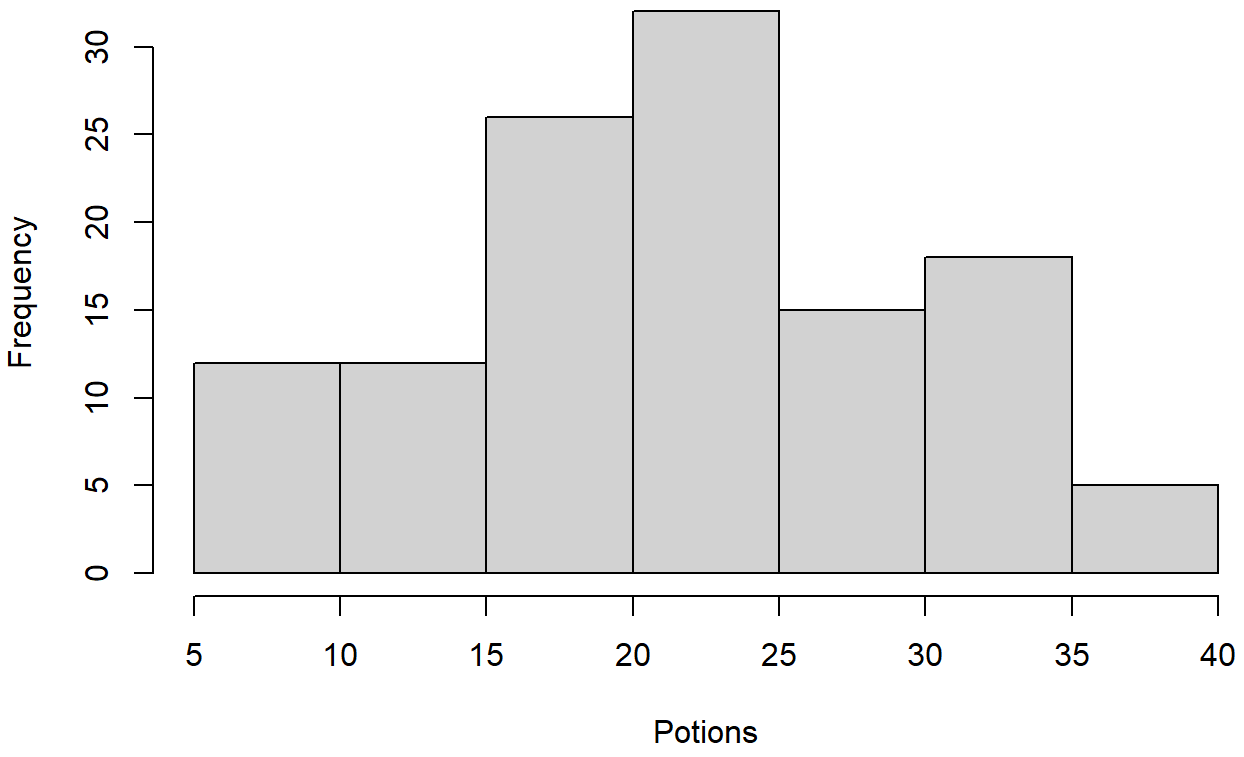
# Write the PokéMart data to a CSV file
## complete the write.csv function ##
write.csv(pokemart_df, "PokemartData.csv", row.names = F)
```

## Instruction 5: Construct histograms

```
# Read PokéMart data
pokemart_data <- read.csv("./PokemartData.csv")

Potions_Plot = qplot(potions, data=pokemart_data, geom="histogram")
ggsave(Potions_Plot,file="Potions_Plot.pdf")


hist(pokemart_data$potions, xlab = "Potions", main = "Distribution of Potions")
```

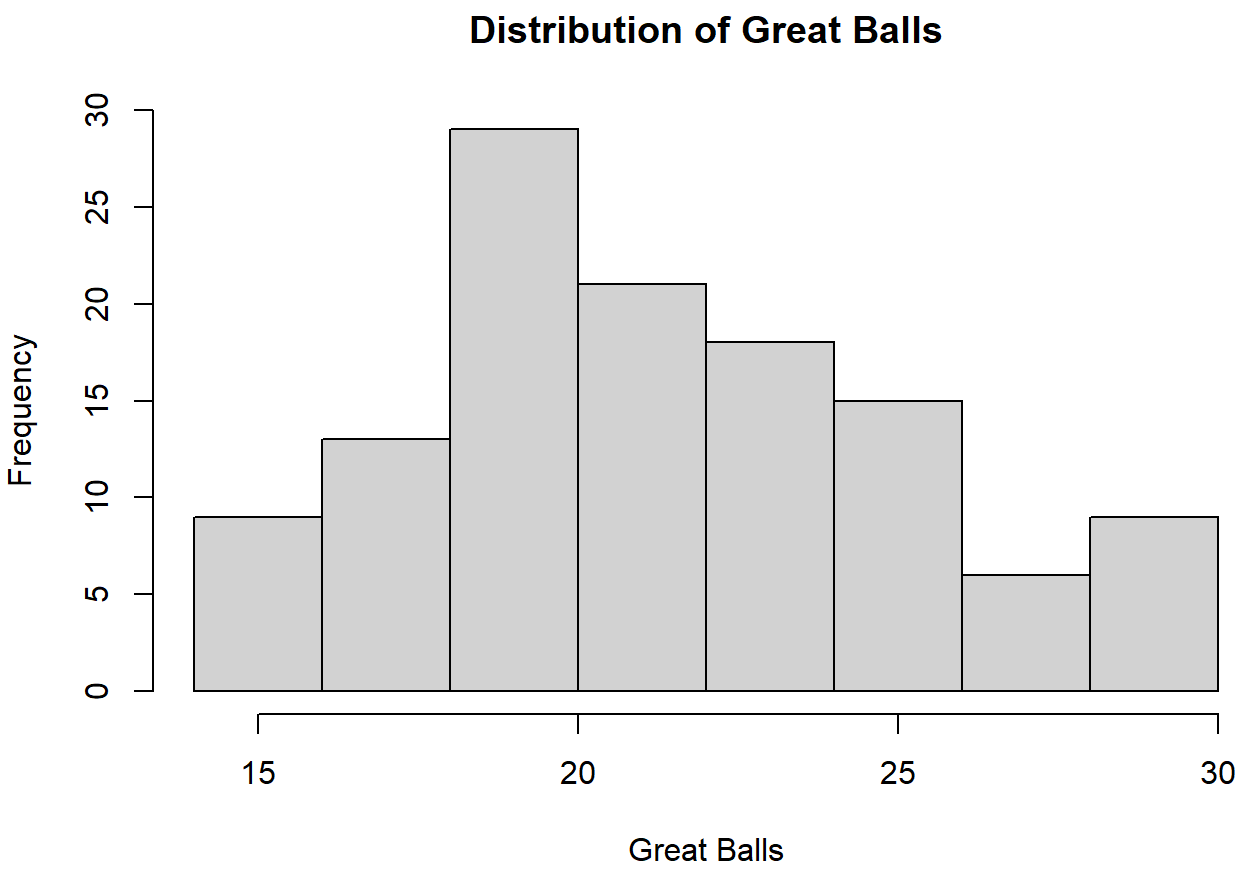



```
## Do the same as above to make histograms for Great Balls and Ultra Balls ##

#Great Balls

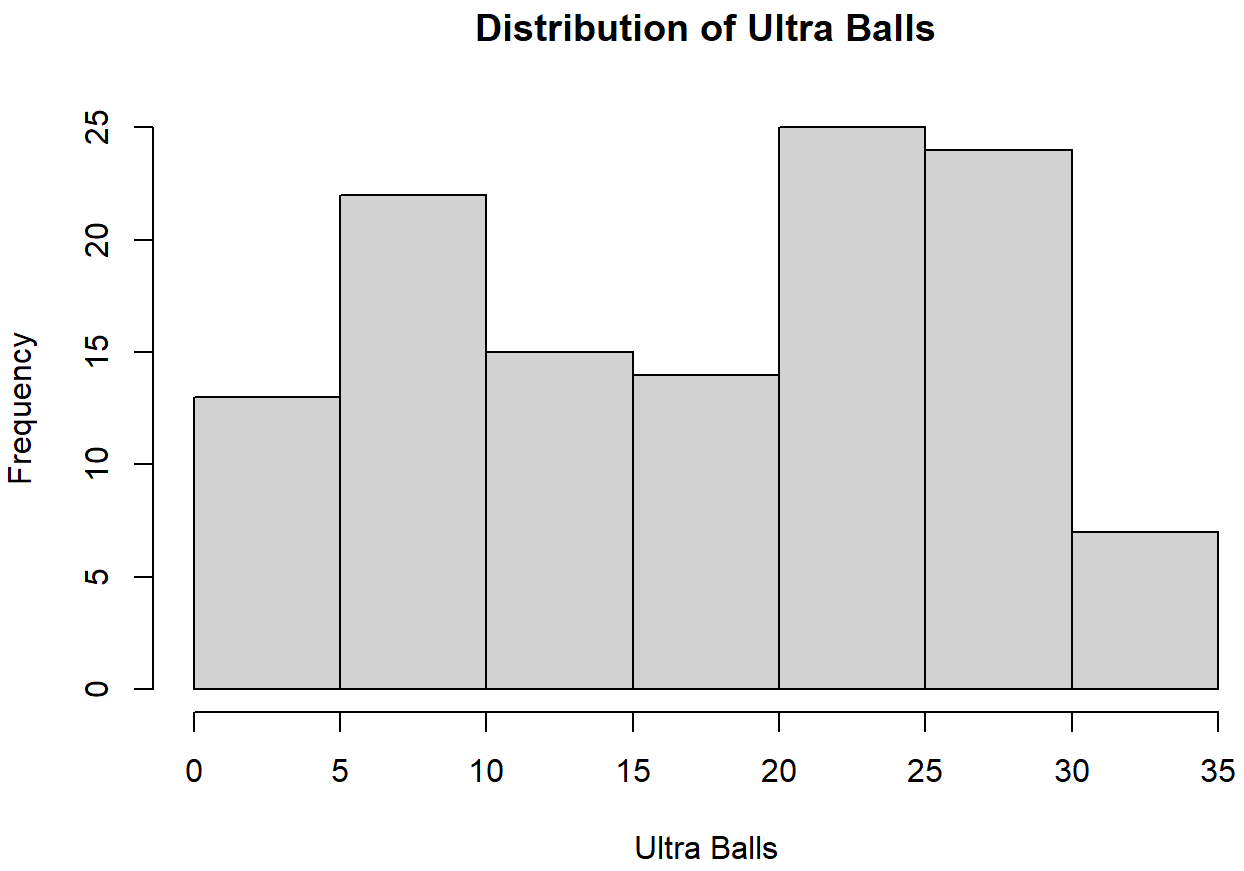
Great_Ball_Plot = qplot(great_balls, data=pokemart_data, geom="histogram")
ggsave(Great_Ball_Plot,file="Great_Ball_Plot.pdf")


hist(pokemart_data$great_balls, xlab = "Great Balls", main = "Distribution of Great Balls")
```




```
#Ultra Balls
Ultra_Ball_Plot = qplot(ultra_balls, data=pokemart_data, geom="histogram")
ggsave(Ultra_Ball_Plot,file="Ultra_Ball_Plot.pdf")


hist(pokemart_data$ultra_balls, xlab = "Ultra Balls", main = "Distribution of Ultra Balls")
```




```r
# Create histograms for potions, Pokéballs, great balls, and ultra balls purchased by trainers
histogram_plot <- ggplot(pokemart_data, aes(x = potions)) +
  geom_histogram(binwidth = 1, fill = "blue", color = "black") +
  ggtitle("Distribution of Potions Purchased")

# Save the histogram as a .png file
ggsave("Potions_Histogram.png", plot = histogram_plot)

## Repeat for Pokéballs, Great Balls, and Ultra Balls ##

#Pokeballs
pokeball_histogram <- ggplot(pokemart_data, aes(x = pokeballs)) +
  geom_histogram(binwidth = 1, fill = "blue", color = "black") +
  ggtitle("Distribution of Pokeballs Purchased")

# Save the histogram as a .png file
ggsave("Pokeballs_Histogram.png", plot = pokeball_histogram)

#Great Balls
histogram_plot3 <- ggplot(pokemart_data, aes(x = great_balls)) +
  geom_histogram(binwidth = 1, fill = "blue", color = "black") +
  ggtitle("Distribution of great balls Purchased")
```

```r
# Save the histogram as a .png file
ggsave("great_balls_Histogram.png", plot = histogram_plot3)

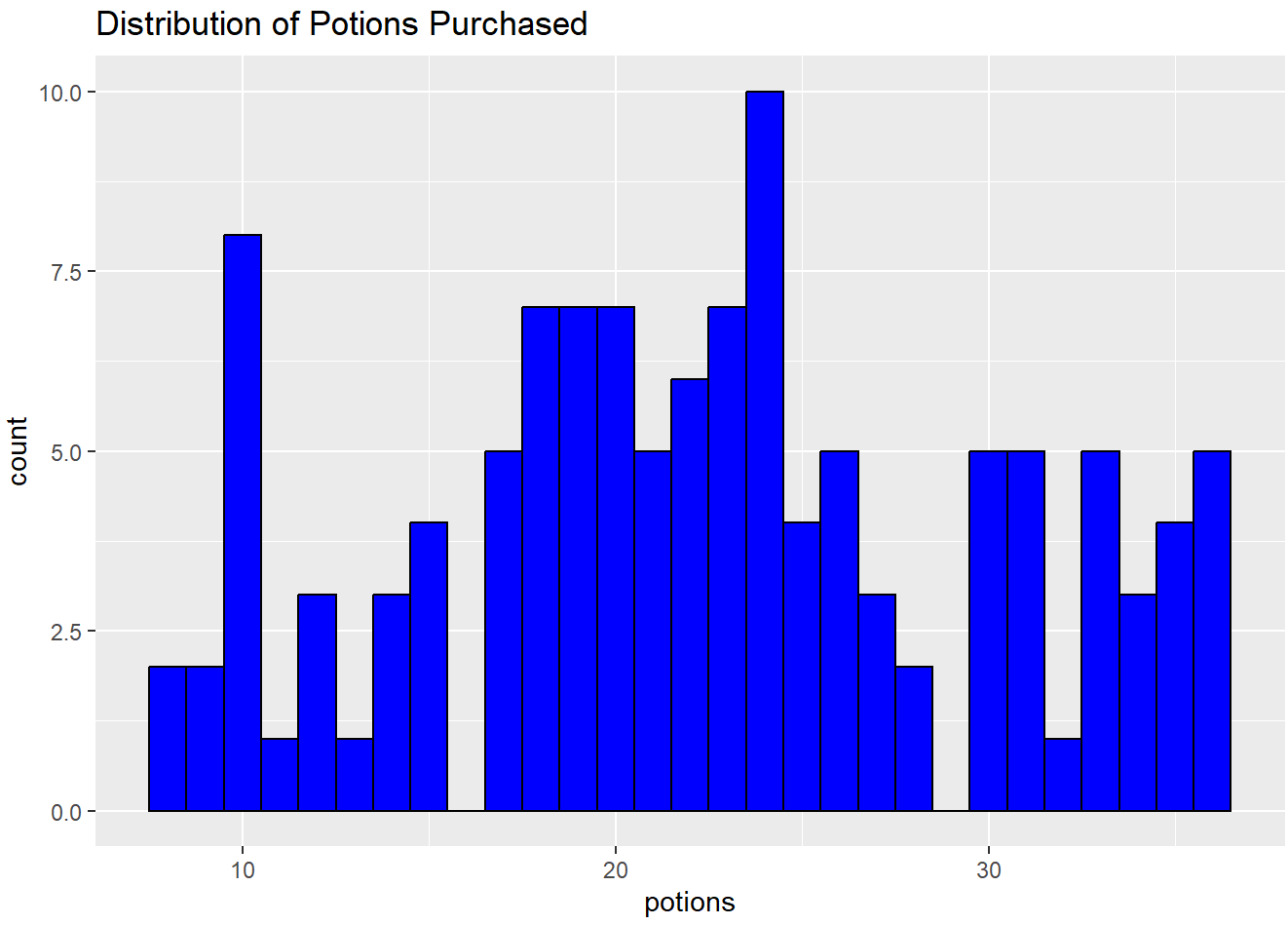
#Ultra balls
histogram_plot4 <- ggplot(pokemart_data, aes(x = ultra_balls)) +
  geom_histogram(binwidth = 1, fill = "blue", color = "black") +
  ggtitle("Distribution of Ultra Balls Purchased")

# Save the histogram as a .png file
ggsave("ultra_balls_Histogram.png", plot = histogram_plot4)

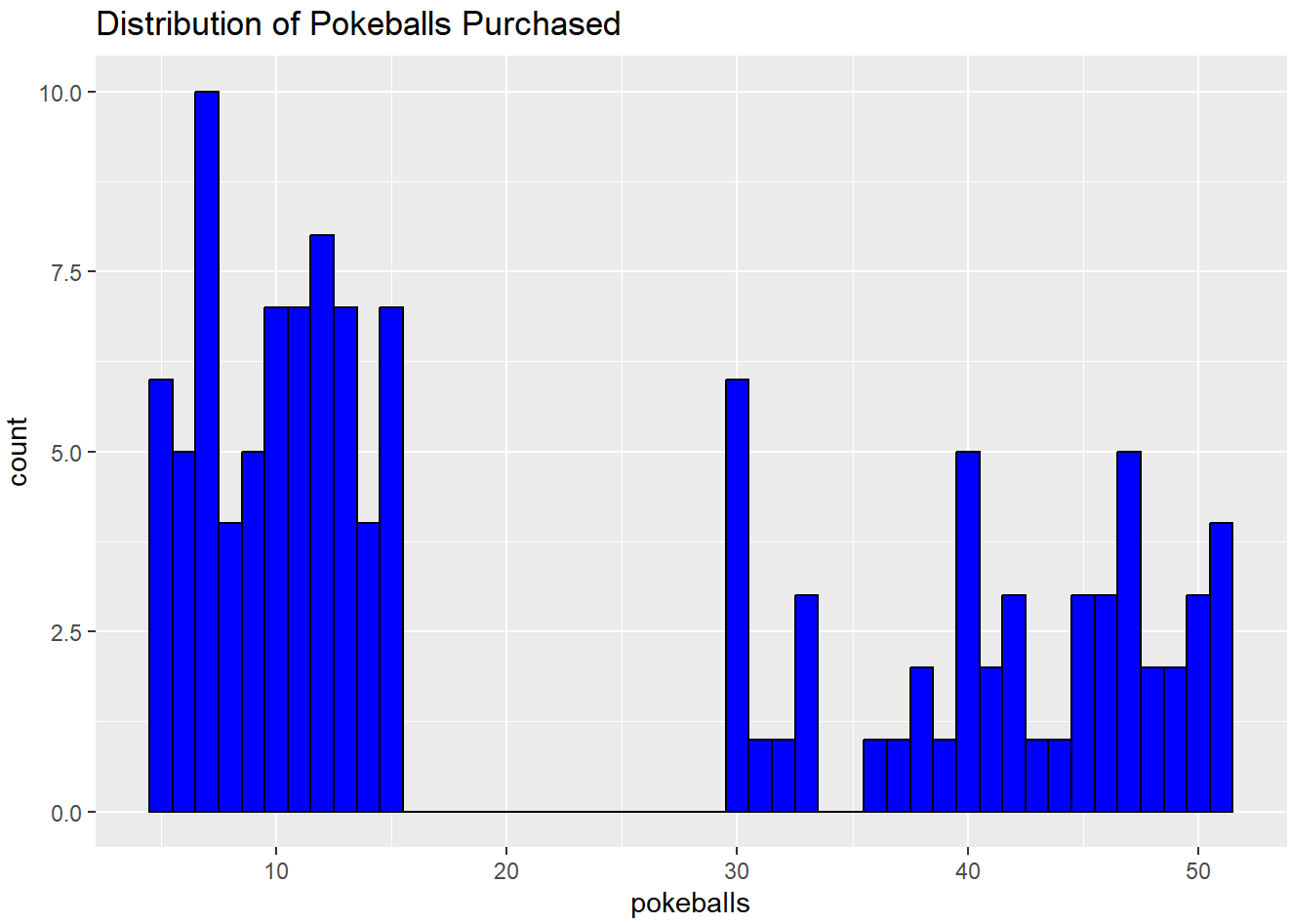


# Scatter plot of Pokémon attack and defense stats
scatter_plot <- ggplot(selected_pokemon, aes(x = attack, y = defense, color = type1)) +
  geom_point(size = 3) +
  ggtitle("Scatter Plot of Pokémon Attack and Defense Stats") +
  labs(color = "Main Type")

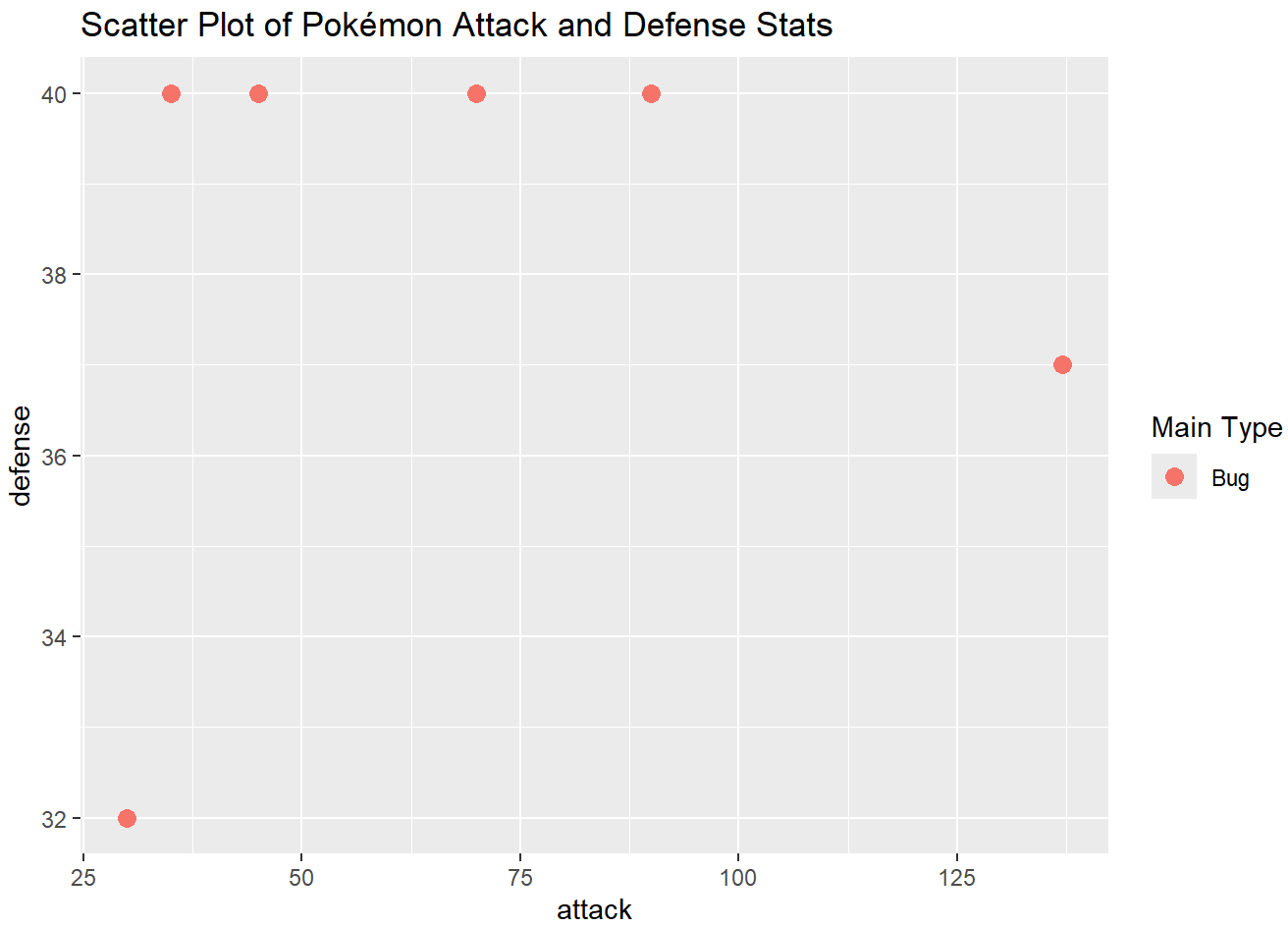
# Save scatter plot as .png file
ggsave("Pokemon_Scatter_Plot.png", plot = scatter_plot)
histogram_plot
```



```
pokeball_histogram
```




```
scatter_plot
```




## Instruction 6: Write three Python functions

The first function calculates the number of legendary Pokemon. The second function calculates the number of gym leaders. The third function calculates the total sales of all Pokemart items.

```
modifiers <- dbReadTable(mydb, "Modifiers")
pokemart_df <- dbReadTable(mydb, "PokeMart")

dbDisconnect(mydb) ## Disconnect database
```

```
[1] TRUE
```

```
import pandas as pd
import math
import random
```

```
# Convert the R dataframe into a Python pandas dataframe
## complete the following code ##
pokemon_df = r.all_pokemon
trainer_df = r.all_trainers
pokemart_df = r.pokemart_df
```

```
modifiers_df = r.modifiers
# Check the first few rows of the dataframe
pokemon_df.head()
```

```
         name  type1   type2  total  ...  sp_defense  speed  generation  legendary
0   Bulbasaur  Grass  Poison    318  ...          65     45           1          0
1     Ivysaur  Grass  Poison    405  ...          80     60           1          0
2    Venusaur  Grass  Poison    525  ...         100     80           1          0
3  Charmander   Fire    None    309  ...          50     65           1          0
4  Charmeleon   Fire    None    405  ...          65     80           1          0

[5 rows x 12 columns]
```

```
# Function 1: Calculate the number of legendary Pokémon by type (both main and secondary types)
def count_legendary_pokemon(df):
    # Filter for legendary Pokémon and group by Type1 and Type2
    type_1_count = df[df['legendary'] == 1].groupby(['type1']).size().reset_index(name='Type_1_Le
    type_2_count = df[df['legendary'] == 1].groupby(['type2']).size().reset_index(name='Type_2_Le
    return type_1_count, type_2_count


# Function 2: Calculate the number of gym leaders
def count_gym_leaders(df):
    gym_leader_count = df[df['gym_leader'] == "Yes"].shape[0]
    return gym_leader_count

# Function 3: Calculate the total sales of PokéMart items
def total_sales(df):
    total_sales_value = sum(df['total_sale'])
    return total_sales_value

# Running the functions with the passed dataframes
legendary_counts = count_legendary_pokemon(pokemon_df)
gym_leader_count = count_gym_leaders(trainer_df)
sales_total = total_sales(pokemart_df)

print("Legendary Pokémon Counts by Type 1:")
```

```
Legendary Pokémon Counts by Type 1:
```

```
print(legendary_counts[0])
```

```
      type1  Type_1_Legendary_Count
0       Bug                       1
1      Dark                       4
2    Dragon                      12
3  Electric                       7
4     Fairy                       3
```

```
5   Fighting                      6
6       Fire                      5
7     Flying                      2
8      Ghost                      3
9      Grass                      4
10    Ground                      4
11       Ice                      3
12    Normal                      4
13    Poison                      2
14   Psychic                     27
15      Rock                      3
16     Steel                      7
17     Water                      9
```

```
print("Legendary Pokémon Counts by Type 2:")
```

```
Legendary Pokémon Counts by Type 2:
```

```
print(legendary_counts[1])
```

```
       type2  Type_2_Legendary_Count
0       Dark                       2
1     Dragon                       7
2   Electric                       1
3      Fairy                       6
4   Fighting                       6
5       Fire                       3
6     Flying                      12
7      Ghost                       5
8      Grass                       4
9     Ground                       3
10       Ice                       4
11   Psychic                       4
12     Steel                       6
13     Water                       2
```

```
print("\nNumber of Gym Leaders:")
```

```
Number of Gym Leaders:
```

```
print(gym_leader_count)
```

```
71
```

```
print("\nTotal Sales of PokéMart Items:")
```

Total Sales of PokéMart Items:

```
print(sales_total)
```

```
5552500
```

## Instruction 7: Calculate stats for each Pokemon at level 100 and identify the strongest Pokemon

```
# Step 1: Identify the data
level_100_multipliers = modifiers_df.loc[99].iloc[1:6]  # Columns 2-6

# Step 2: Select the corresponding columns (5 to 10) from the pokemon dataframe
pokemon_features_to_multiply = pokemon_df.iloc[:, 4:9]  # Columns 5-10

# Step 3: Multiply the relevant columns (element-wise) ## complete Step 3 ##
pokemon_scaled = pokemon_features_to_multiply.mul(level_100_multipliers.values)

# Step 4: Replace the original columns with the scaled values
pokemon_df.iloc[:, 4:9] = pokemon_scaled
```

```
# Step 1: Identify the data
stats_columns = pokemon_df.iloc[:, 4:9]

# Step 2: Calculate the percentile for each stat column ## Complete Step 2 ##
percentiles = stats_columns.rank(pct=True)

# Step 3: Calculate the average percentile for each Pokémon
pokemon_df['average_percentile'] = percentiles.mean(axis=1)

# Step 4: Identify the Pokémon with the highest average percentile
strongest_pokemon = pokemon_df.loc[pokemon_df['average_percentile'].idxmax()]
print(strongest_pokemon)
```

```
name                 Eternamax Eternatus
type1                                Poison
type2                                Dragon
total                                  1125
hp                                     1530
attack                                  345
defense                                 750
sp_attack                               375
sp_defense                              750
speed                                   130
generation                                8
legendary                                 1
```

```
average_percentile              0.958991
Name: 938, dtype: object
```

## Instruction 8: Simulate Pokemon battle

```
# Convert R dataframe to Python (add code)
pokemon_df = r.all_pokemon
trainer_df = r.all_trainers
```

```
## !!!! THIS SECTION IS COMPLETE. NO NEED TO EDIT !!!! ##


#Battle Function

# Function to retrieve Pokémon stats
def get_pokemon(name):
    if pd.isna(name):  # If the trainer has no Pokémon left
        return None
    pokemon = pokemon_df[pokemon_df['name'] == name]
    if pokemon.empty:
        raise ValueError(f"Pokémon '{name}' not found in the data.")
    return pokemon.iloc[0]

# Function to simulate an attack
def attack(attacker, defender, use_special=False):
    if use_special:
        attack_stat = attacker['special_attack']
        defense_stat = defender['special_defense']
    else:
        attack_stat = attacker['attack']
        defense_stat = defender['defense']

    # Calculate damage based on the formula provided
    damage = max(0, attack_stat - math.ceil(defense_stat / 4))
    return damage

# Function to simulate a battle between two Pokémon
def battle(pokemon1, pokemon2):
    hp1 = pokemon1['hp']
    hp2 = pokemon2['hp']

    special1_used = 0
    special2_used = 0

    # Determine who attacks first based on speed
    speed1 = pokemon1['speed']
    speed2 = pokemon2['speed']
```

```
    first_attacker = None
    if speed1 > speed2:
        first_attacker = pokemon1
    else:
        first_attacker = pokemon2

    # Battle loop
    while hp1 > 0 and hp2 > 0:
        if first_attacker is pokemon1:
            # Pokémon 1 attacks first
            damage = attack(pokemon1, pokemon2)
            hp2 -= damage
            if hp2 <= 0:
                return 1  # Pokémon 1 wins

            # Pokémon 2 attacks
            damage = attack(pokemon2, pokemon1)
            hp1 -= damage
            if hp1 <= 0:
                return 2  # Pokémon 2 wins

        else:
            # Pokémon 2 attacks first
            damage = attack(pokemon2, pokemon1)
            hp1 -= damage
            if hp1 <= 0:
                return 2  # Pokémon 2 wins

            # Pokémon 1 attacks
            damage = attack(pokemon1, pokemon2)
            hp2 -= damage
            if hp2 <= 0:
                return 1  # Pokémon 1 wins

# Function to simulate a battle between two trainers
def trainer_battle(trainer1, trainer2):
    # Extract the list of Pokémon for each trainer, dropping NaN values
    team1 = [get_pokemon(pokemon) for pokemon in trainer_df[trainer_df['trainer_id'] == trainer1]
    team2 = [get_pokemon(pokemon) for pokemon in trainer_df[trainer_df['trainer_id'] == trainer2]

    # Each trainer uses the first Pokémon in their team until one faints
    while len(team1) > 0 and len(team2) > 0:
        pokemon1 = team1[0]
        pokemon2 = team2[0]

        # Battle between the first Pokémon
        result = battle(pokemon1, pokemon2)

        # If Pokémon 1 wins, remove Pokémon 2 from Trainer 2's team
        if result == 1:
            team2.pop(0)
```

```
        # If Pokémon 2 wins, remove Pokémon 1 from Trainer 1's team
        else:
            team1.pop(0)

    # Determine the winner based on remaining Pokémon
    if len(team1) > 0:
        return trainer1
    else:
        return trainer2

# Function to run a tournament
def run_tournament(trainers):
    round_num = 1
    while len(trainers) > 1:
        print(f"Round {round_num} begins!")
        random.shuffle(trainers)
        winners = []

        # Match trainers in pairs
        for i in range(0, len(trainers), 2):
            trainer1 = trainers[i]
            trainer2 = trainers[i + 1] if i + 1 < len(trainers) else None

            if trainer2:
                # Run the battle between trainer1 and trainer2
                print(f"{trainer1} vs {trainer2}")
                winner = trainer_battle(trainer1, trainer2)
                print(f"{winner} wins the battle!")
                winners.append(winner)
            else:
                # If odd number of trainers, the last trainer advances automatically
                print(f"{trainer1} advances to the next round (no opponent).")
                winners.append(trainer1)

        trainers = winners
        round_num += 1

    # The last remaining trainer is the champion
    print(f"The champion is: {trainers[0]}!")
    return trainers[0]

# List of trainers
trainers = list(trainer_df['trainer_id'])
```

```
champion = run_tournament(trainers)
```

```
Round 1 begins!
39 vs 27
39 wins the battle!
```

19 vs 85
19 wins the battle!
143 vs 83
143 wins the battle!
13 vs 79
13 wins the battle!
1 vs 159
1 wins the battle!
81 vs 160
81 wins the battle!
101 vs 100
100 wins the battle!
28 vs 87
87 wins the battle!
61 vs 21
61 wins the battle!
152 vs 106
152 wins the battle!
33 vs 148
33 wins the battle!
91 vs 135
135 wins the battle!
50 vs 125
125 wins the battle!
4 vs 59
59 wins the battle!
112 vs 86
86 wins the battle!
60 vs 68
68 wins the battle!
80 vs 6
80 wins the battle!
78 vs 26
26 wins the battle!
53 vs 123
53 wins the battle!
64 vs 88
64 wins the battle!
146 vs 70
70 wins the battle!
109 vs 131
131 wins the battle!
138 vs 155
155 wins the battle!
94 vs 44
44 wins the battle!
98 vs 65
65 wins the battle!
117 vs 77
117 wins the battle!
57 vs 11

11 wins the battle!
119 vs 120
120 wins the battle!
10 vs 84
10 wins the battle!
22 vs 14
14 wins the battle!
76 vs 124
124 wins the battle!
63 vs 95
95 wins the battle!
118 vs 96
96 wins the battle!
129 vs 153
129 wins the battle!
136 vs 115
115 wins the battle!
105 vs 158
158 wins the battle!
127 vs 71
71 wins the battle!
93 vs 58
58 wins the battle!
151 vs 126
151 wins the battle!
154 vs 133
154 wins the battle!
130 vs 69
69 wins the battle!
54 vs 72
72 wins the battle!
31 vs 113
113 wins the battle!
137 vs 97
137 wins the battle!
23 vs 48
48 wins the battle!
92 vs 30
30 wins the battle!
24 vs 74
24 wins the battle!
5 vs 139
5 wins the battle!
110 vs 32
110 wins the battle!
15 vs 116
15 wins the battle!
75 vs 66
66 wins the battle!
41 vs 7
7 wins the battle!

121 vs 73
73 wins the battle!
2 vs 149
2 wins the battle!
134 vs 18
18 wins the battle!
37 vs 51
37 wins the battle!
102 vs 42
102 wins the battle!
142 vs 99
99 wins the battle!
36 vs 114
114 wins the battle!
132 vs 108
108 wins the battle!
46 vs 164
46 wins the battle!
3 vs 20
3 wins the battle!
156 vs 40
40 wins the battle!
107 vs 89
89 wins the battle!
9 vs 144
144 wins the battle!
43 vs 157
43 wins the battle!
162 vs 140
162 wins the battle!
16 vs 128
16 wins the battle!
47 vs 52
47 wins the battle!
38 vs 35
35 wins the battle!
55 vs 122
55 wins the battle!
145 vs 49
145 wins the battle!
56 vs 150
56 wins the battle!
147 vs 104
147 wins the battle!
62 vs 103
103 wins the battle!
25 vs 34
25 wins the battle!
90 vs 161
90 wins the battle!
12 vs 82

82 wins the battle!
67 vs 111
111 wins the battle!
163 vs 17
163 wins the battle!
45 vs 8
45 wins the battle!
29 vs 141
141 wins the battle!
Round 2 begins!
40 vs 26
40 wins the battle!
46 vs 58
58 wins the battle!
64 vs 18
18 wins the battle!
53 vs 86
86 wins the battle!
147 vs 135
135 wins the battle!
99 vs 68
68 wins the battle!
125 vs 5
125 wins the battle!
48 vs 80
48 wins the battle!
163 vs 30
163 wins the battle!
141 vs 95
95 wins the battle!
108 vs 82
82 wins the battle!
151 vs 71
151 wins the battle!
39 vs 13
13 wins the battle!
110 vs 137
110 wins the battle!
16 vs 131
16 wins the battle!
129 vs 33
33 wins the battle!
37 vs 90
90 wins the battle!
24 vs 1
24 wins the battle!
115 vs 2
115 wins the battle!
35 vs 61
35 wins the battle!
69 vs 155

69 wins the battle!
25 vs 10
25 wins the battle!
102 vs 56
56 wins the battle!
47 vs 11
47 wins the battle!
14 vs 44
44 wins the battle!
87 vs 65
87 wins the battle!
70 vs 152
70 wins the battle!
3 vs 100
100 wins the battle!
89 vs 81
81 wins the battle!
145 vs 96
96 wins the battle!
113 vs 19
19 wins the battle!
117 vs 103
103 wins the battle!
15 vs 45
15 wins the battle!
143 vs 154
154 wins the battle!
111 vs 114
114 wins the battle!
59 vs 7
59 wins the battle!
158 vs 66
66 wins the battle!
144 vs 55
144 wins the battle!
43 vs 73
73 wins the battle!
124 vs 72
124 wins the battle!
162 vs 120
120 wins the battle!
Round 3 begins!
15 vs 69
69 wins the battle!
163 vs 95
95 wins the battle!
115 vs 70
115 wins the battle!
35 vs 59
35 wins the battle!
110 vs 48

110 wins the battle!
56 vs 19
19 wins the battle!
13 vs 47
13 wins the battle!
82 vs 103
103 wins the battle!
90 vs 24
90 wins the battle!
86 vs 125
86 wins the battle!
44 vs 33
33 wins the battle!
81 vs 154
154 wins the battle!
68 vs 58
58 wins the battle!
124 vs 73
73 wins the battle!
96 vs 120
120 wins the battle!
40 vs 144
40 wins the battle!
87 vs 100
87 wins the battle!
151 vs 114
114 wins the battle!
66 vs 25
66 wins the battle!
135 vs 18
135 wins the battle!
16 advances to the next round (no opponent).
Round 4 begins!
13 vs 110
13 wins the battle!
115 vs 90
115 wins the battle!
87 vs 58
87 wins the battle!
154 vs 73
73 wins the battle!
19 vs 114
19 wins the battle!
33 vs 135
135 wins the battle!
86 vs 103
103 wins the battle!
40 vs 69
40 wins the battle!
16 vs 66
66 wins the battle!

35 vs 95
35 wins the battle!
120 advances to the next round (no opponent).
Round 5 begins!
66 vs 115
115 wins the battle!
19 vs 120
120 wins the battle!
35 vs 87
35 wins the battle!
40 vs 13
13 wins the battle!
103 vs 135
135 wins the battle!
73 advances to the next round (no opponent).
Round 6 begins!
13 vs 73
73 wins the battle!
135 vs 120
120 wins the battle!
115 vs 35
115 wins the battle!
Round 7 begins!
120 vs 115
115 wins the battle!
73 advances to the next round (no opponent).
Round 8 begins!
115 vs 73
115 wins the battle!
The champion is: 115!